

INTEGRATION PROJECT 5SC26

Ball & Plate Setup

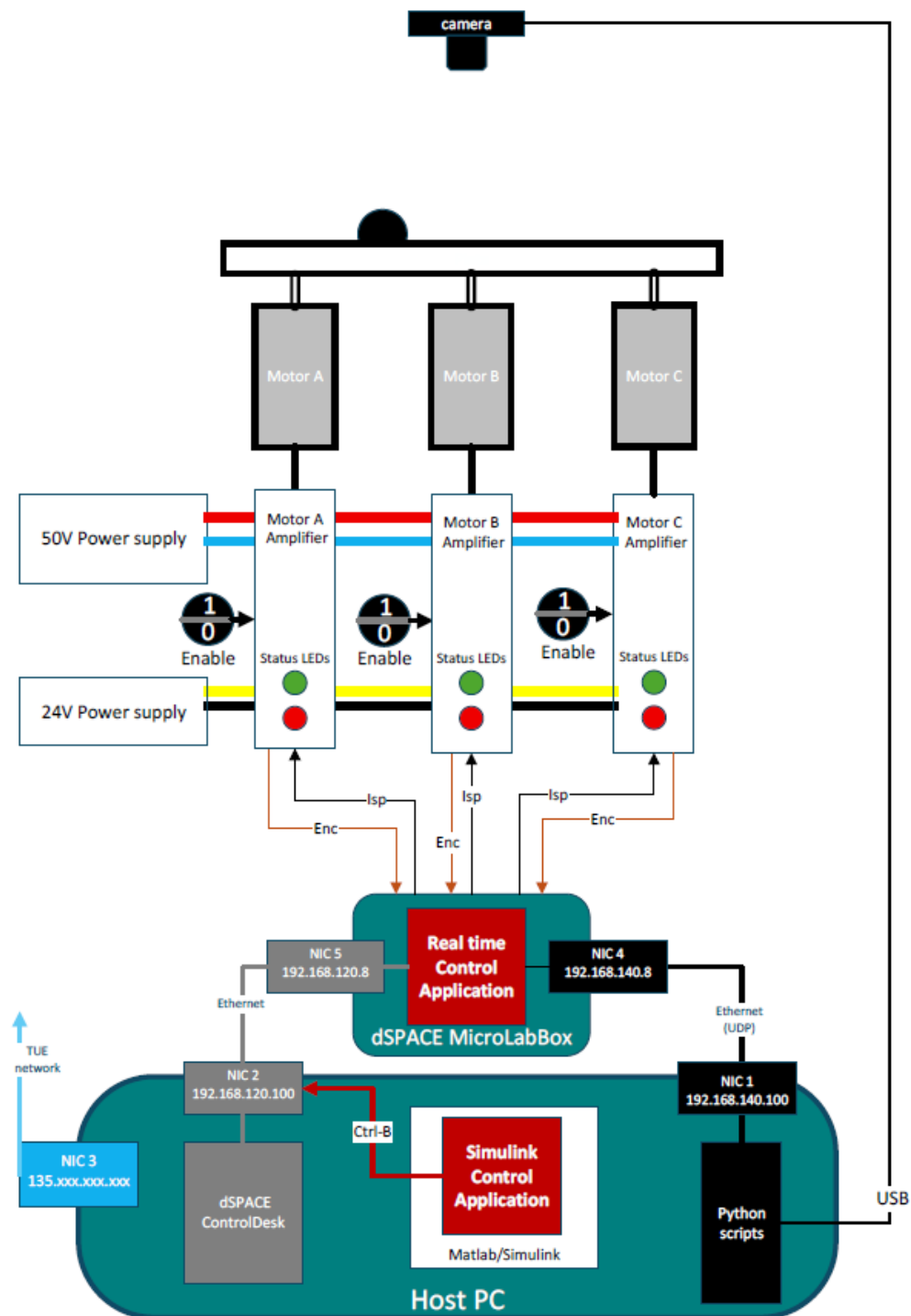
User Guide

May 2026

SYSTEM INTRODUCTION

The figure below shows the main hardware and software components of the Ball&Plate setup and how these components are integrated into an experimental setup for control applications.

A square, white colored plate of size 400x400 mm can be lifted and tilted around its two angles of rotation using three **linear motors** whose sliders are mechanically mounted at the bottom of the plate. A black colored ball can freely roll over the plate, the 2D ball position on the plate can be measured with a **video camera** mounted above the plate.



Every linear motor is powered by its own **amplifier**. All three amplifiers are connected to a 50V DC power supply providing the power for moving the sliders and a 24V DC power supply powering the amplifier's control electronics. The amplifiers will respond to a **motor current setpoint** which will be communicated through a +- 10V analog control signal. Every linear motor has **incremental encoder** that measures the position of the motor's slider. The amplifiers need to be enabled with a **manual switch**, two **status LEDs** indicate normal operation or a fault status.

For every amplifier the corresponding motor's current setpoint (Isp) and the motor's encoder signal (Enc) are interfaced with the dSPACE MicroLabBox (MLBX), the processing unit that runs **the real-time control application**.

The real-time control application running on the MLBX is developed on a **host PC** using **Matlab/Simulink**. The real-time control application is built from a Simulink model through the automated code generation using dSPACE tooling, this code generation is started by pressing **Ctrl-B** from the Simulink model.

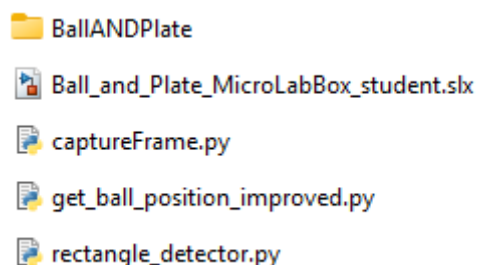
The USB video camera is connected to **Python scripts** on the Host-PC running video processing algorithms to do camera calibration and ball position measurement. The ball position is communicated to the real-time control application using a local network through Network Interface Controller 1 (NIC1) on the Host PC and NIC4 on the MLBX running a UDP protocol.

To communicate with the real-time control application (recording data, tuning parameters, switching modes of operation etc.) dSPACE **ControlDesk** is used. ControlDesk communicates with the real-time application through a separate local network using NIC2 on the Host PC and NIC5 on the MLBX. Finally NIC3 on the HostPC gives connection to the TU/e network.

DOWNLOADING FILES FROM CANVAS

On the CANVAS MODULES pages of the 5SC26 course find the pages with 'Documentation and software ball and plate balancing system'. Here you find:

- [Bal&Plate UserGuide.pdf](#). Which is this manual.
- [dSPACE manual ControlDesk](#). Relevant part is mainly pages 31-47
- [DataSheet PO1 23x80.pdf](#). This is the datasheet of the linear motors that lift and tilt the plate.
- [BallANDPlate student startup files](#). This is a .zip file that you can download to C:\ of the HostPC and extract there. It will give you a folder named BallANDPlate with inside the following:



The folder called BallANDPlate contains all the dSPACE ControlDesk files. The .slx file is your Simulink startup file and the three .py files are the Python scripts used for camera calibration and ball position measurement.

PYTHON SCRIPTS FOR BALL POSITION MEASUREMENT

The following python scripts for ball position measurement are available as starting point:

- captureFrame.py
- rectangle_detector.py
- get_ball_position_improved.py

You can open and run the scripts in **Visual Studio Code**. To setup VS Code to run Python scripts you first have to install the Microsoft official **Python Extension to VS Code**. Next also install the image processing library cv2: **pip install opencv-python**

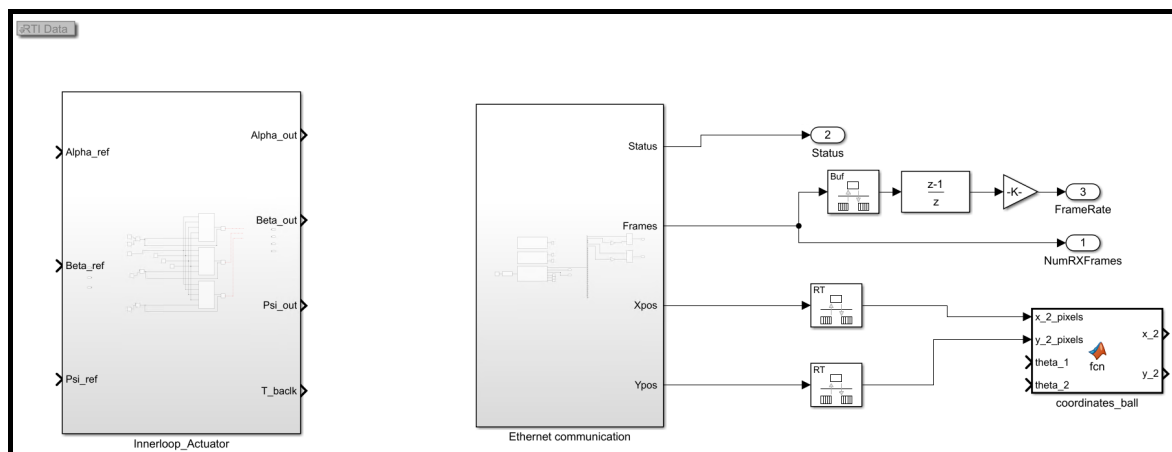
MATLAB-SIMULINK FILES

Use Matlab **2019B**.

The Simulink file that you will get as a starting point is:

- Ball_and_Plate_MicroLabBox_student.slx

The Simulink model looks as follows:



Before you can build code (Ctrl-B) from this you have to specify a value for the variable T_{s_Outer} which is used in the Rate Transition blocks. The Model's Fixed step size is set to 0.001 s you can make the T_{s_Outer} a multiple of this for example 0.1s. So in Matlab command window type:

```
>> Ts_Outer=0.1;
```

Do never delete the block: RTI Data.

dSPACE ControlDesk

The starting point for ControlDesk is a Project called BallANDPlate with inside the experiment called : Experiment_001. There is no Layout defined.

To get started with using ControlDesk ask for an introduction session with lab supervisor.

TYPICAL WORKFLOWS

WORKFLOW A: testing your inner loop controllers is as follows:

The Host PC login and software startup

- 1) Power the Host-PC and login using your TU/e account.
- 2) Copy your working folder from your network (OneDrive) to a folder on the hard drive C:\ on the Host PC.
- 3) Start Matlab/Simulink and open your Simulink model, load variables to Matlab workspace using .mat files or .m scripts.
- 4) Power the MicroLabBox, the power switch is on the back of the box. A green LED on the top of the MLBX will show that it is powered.
- 5) Launch ControlDesk and open your Project and Experiment.
- 6) Build your real time application from Simulink by pressing Ctrl-B
- 7) In ControlDesk press 'Go On Line'

Power on the amplifiers

- 8) Put the amplifier enable switches in the '0' position.
- 9) Switch on the 24V power supply
- 10) Switch the 50V power supply and push the POWER ON/OFF button on the power supply front panel.
- 11) Put the amplifier enable switches in the '1' position. The upper status LED on each amplifier should be green, the lower LED should be off.

Lift the plate to its operating position

- 12) Reset the encoder readings
- 13) Enable your inner loop controller
- 14) Give the operating position as reference, the plate will lift.

Verify your inner loop control performance

- 15) Prepare your time plotter windows in ControlDesk and press 'Start Measuring'
- 16) In ControlDesk activate your testing position reference signals
- 17) Record signals for analysis.

WORKFLOW B: testing your outer loop controller is as follows:

Put plate in operating position

- 1) Take steps 1) till 14) from WORKFLOW A.

Calibrate the camera

- 2) Position the camera such that it can view the whole plate surface and have the camera xy directions aligned with the plate xy directions.
- 3) Remove the ball from the plate!

- 4) Run the python script: **captureFrame.py** it will save an image of the plate in the file imageFrame.png
- 5) Now run python script: **rectangle_detector.py** it will detect the black edges marked by black tape on the plate. From this the exact orientation of the plate and the pixel to mm ratio will be determined.
- 6) Open the python file: **get_ball_position_improved.py** in Visual Studio Code and specify the correction angle resulting from step 5): Angle of bottom line with horizontal: ... degrees
in line 26 angle = ..
- 7) In the Simulink model open the Matlab function: **coordinates_ball** and change the values in lines 7-9 to match the results of the calibration step:

```
y_offset = 242.82;  
x_offset = 295.83;  
conversion factor = 1120.25;
```
- 8) Rebuild (Ctrl-B) your Simulink model.

Put plate in operating position

- 9) Take steps 1) till 14) from WORKFLOW A.

Verify your outer loop control performance

- 1) Prepare your time plotter windows in ControlDesk and press 'Start Measuring'
- 2) In ControlDesk activate your testing position reference signals
- 3) Record signals for analysis.

